

Scalability & Future Plan

Date	04 July 2026
Team ID	ramanharshitha2005@gmail.com
Project Name	OptiCrop - Smart Agricultural Production Optimization Engine
Maximum Marks	1 Mark

Scalability & Future Plan:

This document outlines how the current project solution can be scaled to handle larger user bases, increased data loads, or extended features in the future. It also captures the team's roadmap for enhancing and evolving the project beyond its current state.

Current System Limitations:

S.No	Limitation	Impact	Priority to Address (High/Medium/Low)
1	Python built-in ThreadingHTTPServer is not production-grade. Suitable for local/LAN use with ~20 concurrent users but not for high-traffic public deployment.	Replacing with gunicorn + Nginx would enable production-scale deployment.	High
2	Flat-file storage (CSV, JSON) does not support dynamic data insertion, queries, or farmer history tracking.	Migrating to PostgreSQL or MongoDB would enable farmer profiles, history, and region-specific analytics.	High
3	Crop recommendation dataset is static (2200 rows from Kaggle). No real-time soil sensor data ingestion.	IoT sensor integration would enable real-time, location-specific recommendations.	High
4	English-only interface. Inaccessible to farmers who are not English-literate.	Adding Telugu, Hindi, and Tamil language support would dramatically expand reach.	High
5	Image disease CNN model requires the full PlantVillage dataset (~5 GB) for training. Lightweight pixel-feature RF fallback has lower accuracy.	Hosting a pre-trained CNN model on cloud inference (AWS SageMaker or Vertex AI) would remove local training dependency.	Medium

Scalability Plan:

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	Python ThreadingHTTPServer handles ~20 concurrent local users. Not suitable for public internet traffic.	Replace ThreadingHTTPServer with gunicorn (WSGI) + Nginx reverse proxy. Deploy on a VPS (Render, Railway, or AWS EC2). Add horizontal scaling via Docker containers if needed.
2	Data Storage	Static CSV and JSON flat files. No dynamic writes or farmer data persistence.	Migrate to PostgreSQL for structured crop/farmer data. Add a REST API layer with SQLAlchemy ORM. Use Redis for caching frequent analytics queries.
3	Performance	Single-core CPU inference for all ML models. No GPU acceleration at inference time.	Deploy ML models on AWS SageMaker endpoints or Google Vertex AI for GPU-accelerated batch inference. Cache frequent /api/analytics responses in Redis.

4	Security	No authentication. Directory traversal prevented by <code>safe_resolve()</code> . No HTTPS.	Add JWT-based authentication for researcher/admin roles. Deploy with HTTPS (Let's Encrypt). Implement API rate limiting per IP address.
---	----------	---	---

Future Roadmap:

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 2	Multilingual interface (Telugu, Hindi, Tamil)	Q3 2026	Expands reach to 10x more farmers; reduces language literacy barrier
Phase 2	GPS-based automatic location detection for weather	Q3 2026	Eliminates manual city name entry; improves accuracy for rural users
Phase 3	IoT soil sensor integration (real-time NPK from sensors)	Q4 2026	Enables live soil monitoring without manual input; continuous recommendations
Phase 3	District and season-specific crop recommendation model	Q4 2026	Hyper-localised recommendations per Indian district and growing season
Phase 4	Crop yield prediction (regression model on historical data)	Q1 2027	Farmers can estimate expected yield in quintals/acre before planting decision
Phase 4	Market price API integration (Agmarknet / e-NAM)	Q1 2027	Recommends most profitable crop based on yield potential + current market price
Phase 4	Native Android / iOS mobile app (offline-capable)	Q2 2027	Offline crop recommendation and disease detection for low-connectivity rural areas